

SEARCHING

MATA KULIAH : STRUKTUR DATA & ALGORITMA

04

TOPIK

- Sequential Search
- Binary Search
- Interpolation Search

PENCARIAN (SEARCHING)

- Pada suatu data seringkali dibutuhkan pembacaan kembali informasi (**information retrieval**) dengan cara searching.
- Searching adalah **proses pencarian data** yang ada pada suatu deret data dengan cara menelusuri data-data tersebut.
- Tahapan paling penting pada searching: **memeriksa jika data yang dicari sama dengan data yang ada pada deret data.**

DASAR ALGORITMA PENCARIAN

1. Input x (x =data yang dicari)
2. Bandingkan x dengan deret data
3. Jika ada data yang sama : lakukan perintah jika data ditemukan (contoh : cetak pesan “Ada”)
4. Jika tidak ada data yang sama : lakukan perintah jika data tidak ditemukan (contoh : cetak pesan “Tidak Ada”)

ALGORITMA PENCARIAN

- Sequential Search
- Binary Search
- Interpolation Search

SEQUENTIAL SEARCH

- Disebut juga **Linear Search** atau metode **pencarian beruntun**.
- Adalah suatu teknik pencarian data yang akan **menelusuri tiap elemen pada suatu kelompok data satu per-satu dari awal sampai akhir**.
- Kelompok data **tidak harus dalam kondisi terurut**.

ALGORITMA SEQUENTIAL SEARCH

1. Input x (x =data yang dicari)
2. Bandingkan x dengan data **ke i sampai ke n**
3. Jika **ada data yang sama** : lakukan perintah jika data ditemukan (contoh : cetak pesan “Ada”)
4. Jika **tidak ada data yang sama** : lakukan perintah jika data tidak ditemukan (contoh : cetak pesan “Tidak Ada”)

ILUSTRASI SEQUENTIAL SEARCH

- Misalnya terdapat array satu dimensi sebagai berikut:

0	1	2	3	4	5	6	7	indeks
8	10	6	-2	11	7	1	100	value

- Kemudian program akan meminta data yang akan dicari, misalnya 6 ($x = 6$).
- Iterasi :
 - $6 = 8$ (tidak!)
 - $6 = 10$ (tidak!)
 - $6 = 6$ (Ya!) => output : “Ada” pada index ke-2
- Jika sampai data terakhir tidak ditemukan data yang sama maka output : “ data yang dicari tidak ada”.

QUESTION

- **Problem:** Apakah cara di atas efisien? Jika datanya ada 10000 dan semua data dipastikan unik?
- **Solution:** Untuk meningkatkan efisiensi, seharusnya jika data yang dicari sudah ditemukan maka perulangan harus dihentikan!
 - Hint: Gunakan break!
- **Question:** Bagaimana cara menghitung ada berapa data dalam array yang tidak unik, yang nilainya sama dengan data yang dicari oleh user?
 - Hint: Gunakan variabel counter yang nilainya akan selalu bertambah jika ada data yang ditemukan!

SEQUENTIAL SEARCH DENGAN SENTINEL

- Perhatikan array satu dimensi sebagai berikut:

0	1	2	3	4	5	6	indeks
3	12	9	-4	21	6		value

- Terdapat 6 buah data dalam array (dari indeks 0 s/d 5) dan terdapat 1 indeks array tambahan (indeks ke 6) yang belum berisi data (disebut sentinel)
- Array pada indeks ke 6 berguna untuk menjaga agar indeks data berada pada indeks 0 s/d 5 saja. Bila pencarian data sudah mencapai array indeks yang ke-6 maka berarti data TIDAK ADA, sedangkan jika pencarian tidak mencapai indeks ke-6, maka data ADA.

BEST AND WORST CASE

- **Best Case** : jika data yang dicari terletak di depan sehingga waktu yang dibutuhkan minimal.
 - **Worst Case** : jika data yang dicari terletak di akhir sehingga waktu yang dibutuhkan maksimal.
 - Contoh :
 - **DATA** = 5 6 9 2 8 1 7 4
 - **Best Case** ketika $x = 5$
 - **Worst Case** ketika $x = 4$
- * x = key/data yang dicari

LATIHAN

- Buatlah flowchart untuk algoritma Sequential Search dan Sequential Search dengan Sentinel
- Buatlah Pseducode untuk algoritma Sequential Search dan Sequential Search dengan Sentinel

BINARY SEARCH

- Teknik pencarian dimana **data dibagi menjadi dua bagian** untuk setiap kali proses pencarian.
- **Data awal harus dalam kondisi terurut.** Sehingga harus dilakukan proses sorting terlebih dahulu untuk data awal.
- Mencari posisi tengah :

$$\text{Posisi tengah} = (\text{posisi awal} + \text{posisi akhir}) / 2$$

ALGORITMA BINARY SEARCH

1. Data diambil dari posisi awal 1 dan posisi akhir N
2. Kemudian cari posisi data tengah dengan rumus: $(\text{posisi awal} + \text{posisi akhir}) / 2$
3. Kemudian data yang dicari dibandingkan dengan data yang di tengah, apakah sama atau lebih kecil, atau lebih besar?
4. Jika data sama, berarti ketemu.
5. Jika lebih besar, maka ulangi langkah 2 dengan posisi awal adalah posisi tengah + 1
6. Jika lebih kecil, maka ulangi langkah 2 dengan posisi akhir adalah posisi tengah - 1

ILUSTRASI BINARY SEARCH

- Contoh Data:
- Misalnya data yang dicari 17

1	2	3	4	5	6	7	8
39	11	12	15	17	23	31	35
A			B				C

- Karena $17 > 15$ (data tengah), maka: awal = tengah + 1

1	2	3	4	5	6	7	8
39	11	12	15	17	23	31	35
			A	B			C

ILUSTRASI BINARY SEARCH

- Karena $17 > 15$ (data tengah), maka: awal = tengah + 1

1	2	3	4	5	6	7	8
39	11	12	15	17	23	31	35
		A	B			C	

- Karena $17 < 23$ (data tengah), maka: akhir = tengah - 1

1	2	3	4	5	6	7	8
39	11	12	15	17	23	31	35
				A=B=C			

- Karena $17 = 17$ (data tengah), maka KETEMU!

BEST AND WORST CASE

- **Best Case** : jika data yang dicari terletak di posisi tengah.
- **Worst Case** : jika data yang dicari tidak ditemukan.
- Contoh :
 - **DATA = 5 6 9 2 8 1 7 4 3**
 - **Best Case** ketika $x = 8$ ($T(n)=1$)
 - **Worst Case** ketika $x = 25$ ($T(n) = 5$ atau $n/2$)
 - $*x = \text{key/data yang dicari}$

LATIHAN

- Buatlah flowchart untuk algoritma Binary Search
- Buatlah Pseducode untuk algoritma Binary Search

INTERPOLATION SEARCH

- Teknik ini dilakukan pada data yang **sudah terurut**.
- Teknik searching ini dilakukan dengan **perkiraan letak data**.
 - Contoh ilustrasi: jika kita hendak mencari suatu nama di dalam buku telepon, misal yang berawalan dengan huruf T, maka kita tidak akan mencarinya dari awal buku, tapi kita langsung membukanya pada $\frac{2}{3}$ atau $\frac{3}{4}$ dari tebal buku.
- Rumus posisi relatif kunci pencarian dihitung dengan rumus:

$$Posisi = \frac{kunci - data[low]}{data[high] - data[low]} \times (high - low) + low$$

- Jika **$data[posisi] > data$** yg dicari, **$high = pos - 1$**
- Jika **$data[posisi] < data$** yg dicari, **$low = pos + 1$**

KONDISI BERHENTI

- Ada 2 kondisi berhenti :
 - jika data ditemukan
 - jika data tidak ditemukan, sampai data[low] lebih besar dari data yang dicari.

KASUS

- Misal terdapat data sbb :

Carilah kode 88 dengan menggunakan cara Interpolation Search ?

Kode	Judul Buku	Pengarang
25	The C++ Programming	James Wood
34	Mastering Delphi 6	Marcopolo
41	Professional C#	Simon Webe
56	Pure JavaScript v2	Michael Bolton
63	Advanced JSP & Servlet	David Dunn
72	Calculus Make it Easy	Gunner Christian
88	Visual Basic 2005 Express	Antonie
96	Artificial Life : Volume 1	Gloria Virginia

PENYELESAIAN

- Kunci Pencarian ? 88
- Low ? 0
- High ? 7
- Posisi = $(88 - 25) / (96 - 25) * (7 - 0) + 0 = [6]$
- Kunci[6] = kunci pencarian, data ditemukan : Visual Basic 2005, Antonie

PENYELESAIAN

- Kunci Pencarian ? 60
- Low ? 0 High ? 7
- Posisi = $(60 - 25) / (96 - 25) * (7 - 0) + 0 = [3]$
- Kunci[3] < kunci pencarian, maka teruskan
- Low = $3 + 1 = 4$
- High = 7
- Ternyata Kunci[4] adalah 63 yang lebih besar daripada 60.
- Berarti tidak ada kunci 60.

LATIHAN

- Lakukan pencarian dengan Interpolation Search dari deret data berikut :

39 11 12 15 17 23 31 35

- $x = 15 \Rightarrow$ low? high?
- $x = 5 \Rightarrow$ low? high?
- $x = 35 \Rightarrow$ low? high?

LATIHAN

- Buatlah flowchart untuk algoritma Interpolation Search
- Buatlah Pseducode untuk algoritma Interpolation Search



END OF SLIDE —
SEARCHING